

PROJECT REPORT
OF
CO-OP PROJECT AT INDUSTRY (MODULE-2)
ON
SHOUVAIR E-COMMERCE WEBSITE

submitted in partial fulfilment of the requirements for the award of degree of
BACHELOR OF ENGINEERING

In
COMPUTER SCIENCE AND ENGINEERING



Submitted by:

Aarav Shourie (2210990010)
Dhruv Kundrai (2210990276)
Aarya Rishi (2210990012)
Advitya Hans (2210990076)

Supervised By:

Dr. Gurpreet Singh
Associate Professor
Chitkara University

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND TECHNOLOGY
CHITKARA UNIVERSITY, PUNJAB, INDIA

DECLARATION

I hereby certify that the work which is being presented in the project report entitled "SHOUVAIR E-COMMERCE WEBSITE" in partial fulfilment of requirement for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of my own work carried out under the supervision of Gurpreet. The matter presented in this project report has not been submitted in any other university/institute for the award of any degree.

Place: Rajpura **(Aarav Shourie)**

Date: (University Roll No. 2210990010)

Place: Rajpura **(Dhruv Kundrai)**

Date: (University Roll No. 2210990276)

Place: Rajpura **(Aarya Rishi)**

Date: (University Roll No. 2210990012)

Place: Rajpura **(Advitya Hans)**

Date: (University Roll No. 2210990076)

This is to certify that the above statement made by the candidates is correct to the best of my knowledge and belief.

Dr. Gurpreet Singh

Associate Professor

Department of Computer Science and Engineering

Chitkara University, Punjab, India

ACKNOWLEDGEMENT

The successful completion of this project would not have been possible without the support, guidance, and encouragement of many individuals, and we take this opportunity to express our heartfelt gratitude to all of them.

We are deeply grateful to our mentor, Dr. Gurpreet Singh, Associate Professor, for their invaluable guidance, consistent support, and expert advice throughout the duration of this CO-OP project. Their insightful feedback and patient mentorship shaped the direction and quality of this work significantly.

We extend our sincere thanks to the management and technical team at Chitkara University for providing us with a stimulating work environment, real-world project experience, and the technical resources necessary to complete this project. The exposure to industry practices during this internship has been immensely valuable to our professional growth.

We are also grateful to the Department of Computer Science and Engineering, Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, for structuring the CO-OP programme in a manner that bridges academic learning with industry requirements.

Our sincere appreciation goes to all our faculty members who laid a strong academic foundation that equipped us to handle this industrial project effectively.

Finally, we owe our deepest gratitude to our families and friends for their unwavering moral support, patience, and encouragement throughout this journey.

Aarav Shourie (2210990010)

Dhruv Kundrai (2210990276)

Aarya Rishi (2210990012)

Advitya Hans (2210990076)

CONTENTS

S.No.	Title	Page No.
	Declaration	ii
	Acknowledgement	iii
	List of Figures and Tables	iv
	Abstract	v
1	Introduction	1
1.1	Background and Motivation	2
1.2	Problem Statement	3
1.3	Objectives of the Project	4
1.4	Scope of the Project	5
2	Methodology	6
2.1	Development Approach	6
2.2	Project Planning and Timeline	8
2.3	System Architecture	9
3	Tools and Technologies	12
3.1	Backend Technologies	12
3.2	Frontend Technologies	14
3.3	Testing Tools	15
3.4	Development and Collaboration Tools	16
4	Implementation	17
4.1	Module 1: User Authentication	17
4.2	Module 2: Product Catalog	19
4.3	Module 3: Shopping Cart	21
4.4	Module 4: Order Management	23
4.5	Module 5: Admin Panel	25
4.6	Security Implementation	26
5	Major Findings / Outcomes / Results	28
5.1	Functional Outcomes	28
5.2	Testing Results	30
5.3	Performance Metrics	32

5.4	Learnings from Industry	33
6	Conclusion and Future Scope	35
6.1	Conclusion	35
6.2	Future Scope	36
	References	38
	Appendices	40

LIST OF FIGURES AND TABLES

List of Figures

Figure No.	Caption	Page No.
2.1	Waterfall + Agile Hybrid Development Methodology	7
2.2	High-Level MVC Architecture of Shouvair	9
2.3	Request-Response Flow Diagram	10
2.4	Entity-Relationship (ER) Diagram	11
4.1	User Registration and Login Flow	18
4.2	JWT Authentication Flow	19
4.3	Product Catalog Page Screenshot	20
4.4	Shopping Cart Interface Screenshot	22
4.5	Order Checkout Flow	24
4.6	Admin Dashboard Screenshot	25
4.7	Helmet.js Security Headers Response	27
5.1	Jest Test Suite Output	31
5.2	Puppeteer E2E Test Flow	31
5.3	Server Response Time Metrics	32

List of Tables

Table No.	Caption	Page No.
2.1	Project Timeline / Sprint Plan	8
3.1	NPM Dependencies Used in Shouvair	13
3.2	Testing Tools and Their Roles	15
4.1	API Endpoints – Authentication Module	19
4.2	API Endpoints – Product Module	21
4.3	API Endpoints – Cart and Order Modules	23
5.1	Test Case Summary	30
5.2	Performance Metrics Comparison	32
5.3	Skills Acquired During CO-OP	34

ABSTRACT

The Shouvair E-Commerce Website is a full-stack web application developed as part of the CO-OP Project at Industry (Module-2) under the Bachelor of Engineering programme in Computer Science and Engineering at Chitkara University. The project involved the design, development, testing, and deployment of a complete online retail platform for footwear products, providing hands-on industry experience in modern web application development.

The application is built on the Node.js runtime using the Express.js framework for server-side routing and middleware management. MongoDB, accessed through the Mongoose ODM, serves as the NoSQL database for flexible and scalable data persistence. EJS (Embedded JavaScript) is employed as the server-side templating engine for dynamic HTML rendering. The system architecture follows the MVC (Model-View-Controller) pattern, ensuring a clean separation of concerns and maintainability.

Key functional modules include: secure user authentication with bcrypt password hashing and JWT-based session tokens, a dynamic product catalog with category and price filters, a shopping cart system, an order placement and history workflow, and a basic admin panel for product management. Security is enforced through the Helmet.js middleware (HTTP header hardening), CORS policy management, and httpOnly cookie storage for authentication tokens.

The application was tested using Jest (unit and integration tests), Supertest (HTTP route assertions), Puppeteer (end-to-end browser automation), and MongoDB Memory Server (isolated database testing). A total of 53 test cases were executed with a 100% pass rate and approximately 92% code coverage. The complete source code is version-controlled at <https://github.com/dhruv276/shouvair>.

This project provided significant industry exposure in full-stack JavaScript development, RESTful API design, database schema modeling, application security, and collaborative development using Git workflows. The result is a functional, secure, and scalable e-commerce platform that serves as a foundation for production deployment with additional enhancements.

Keywords: E-Commerce, Node.js, Express.js, MongoDB, EJS, JWT Authentication, bcrypt, MVC Architecture, REST API, Full-Stack Web Development, Helmet.js, Automated Testing

1. INTRODUCTION

The rapid growth of digital commerce over the past decade has fundamentally altered how consumers discover and purchase products. E-commerce—the online buying and selling of goods and services—has evolved from a niche channel to the dominant mode of retail in many product categories. This transformation has been accelerated by widespread smartphone adoption, affordable internet connectivity, and the emergence of mature web development platforms and frameworks that make building feature-rich online stores increasingly accessible.

Shouvair is a purpose-built e-commerce platform designed for footwear retail. The name is inspired by a combination of 'shoe' and 'savoir-faire'—French for expertise or know-how—reflecting the platform's ambition to deliver an expert, elegant, and seamless shopping experience for footwear enthusiasts. The platform was conceptualised and developed during the CO-OP industrial training period by a team of four Computer Science and Engineering students from Chitkara University.

1.1 Background and Motivation

India's e-commerce sector has experienced exponential growth, driven by the proliferation of smartphones and affordable 4G/5G internet access. The footwear segment represents a sizeable and growing share of online retail in India. Despite the presence of major marketplace platforms such as Amazon, Flipkart, and Myntra, there remains a significant gap for independent footwear brands seeking a dedicated, branded online presence without surrendering control over customer data, brand identity, and profit margins to third-party platforms.

The CO-OP programme at Chitkara University provided the team with the opportunity to work on a real-world industry project that addresses this gap. The technical challenge of building a complete e-commerce system—covering user management, product cataloguing, cart management, order processing, security, and automated testing—served as an ideal context for applying the theoretical knowledge acquired during the undergraduate programme to practical, production-grade software development.

1.2 Problem Statement

Small and medium footwear businesses lack affordable, customizable e-commerce solutions that they fully own and control. Existing marketplace platforms impose high commission fees (8–15%),

restrict branding, limit access to customer data, and create dependency on third-party policies. Open-source alternatives like WooCommerce are often built on dated PHP/MySQL stacks with performance limitations. There is a need for a modern, Node.js-based, full-stack e-commerce solution that a footwear business can self-host, customize, and scale.

The Shouvair project directly addresses this need by delivering a complete, open-source, Node.js/MongoDB e-commerce platform with industry-standard security, comprehensive automated testing, and clear documentation—built using entirely free and open-source technologies.

1.3 Objectives of the Project

The primary objectives of the Shouvair project are:

1. To design and implement a full-stack e-commerce web application for footwear retail using Node.js, Express.js, MongoDB, and EJS.
2. To implement a secure user authentication system using bcrypt for password hashing and JSON Web Tokens (JWT) for session management.
3. To develop a dynamic product catalog with search and filter capabilities.
4. To build a functional shopping cart and order management system.
5. To integrate production-grade security middleware including Helmet.js and CORS configuration.
6. To implement a comprehensive automated testing suite using Jest, Supertest, and Puppeteer.
7. To gain hands-on industry experience in full-stack web development, collaborative Git workflows, and software project documentation.

1.4 Scope of the Project

The Shouvair project encompasses the following functional areas:

- User authentication: registration, login, secure session management, and logout.
- Product catalog: dynamic listing, individual product detail pages, category and price filtering.

- Shopping cart: add, update quantity, remove items, and clear cart.
- Order management: checkout, order placement, and order history for authenticated users.
- Admin panel: product CRUD (Create, Read, Update, Delete) operations.
- Security: bcrypt password hashing, JWT tokens, Helmet.js HTTP headers, CORS, and dotenv configuration management.
- Testing: unit, integration, and end-to-end automated test suites.

The following features are outside the scope of the current version and are identified as future enhancements: real-time payment gateway integration, email notifications, cloud image storage, mobile application, and multi-vendor marketplace support.

2. METHODOLOGY

This chapter describes the development methodology adopted for the Shouvair project, including the software development lifecycle model, project planning approach, and the high-level system architecture that guided the implementation.

2.1 Development Approach

The Shouvair project followed a hybrid development methodology combining elements of the Waterfall model for structured phase completion (requirements → design → implementation → testing → documentation) with an Agile-inspired iterative approach for feature development. This hybrid approach was chosen because:

- The academic submission deadline required a structured phase plan (Waterfall discipline).
- The team was building features iteratively, with each module (auth, products, cart, orders) being developed, tested, and integrated independently before moving to the next (Agile discipline).
- The CO-OP industry environment required flexibility to incorporate mentor feedback between iterations.

2.1.1 Phase 1 – Requirements Analysis (Week 1)

The team conducted a requirements elicitation session with the industry mentor to identify the core features of the Shouvair platform. Functional requirements (FR) and non-functional requirements (NFR) were documented. A competitive analysis of existing e-commerce platforms (Amazon, Shopify, WooCommerce) was performed to identify feature benchmarks and technology choices.

2.1.2 Phase 2 – System Design (Week 2)

Based on the requirements, the system architecture was designed. This phase produced: the MVC architectural diagram, MongoDB collection schemas (User, Product, Cart, Order), Express.js route structure, middleware pipeline design, and EJS template hierarchy. The directory structure was scaffolded and the project was initialized with npm and version-controlled on GitHub.

2.1.3 Phase 3 – Implementation (Weeks 3–8)

Implementation proceeded module by module in the following sequence: (1) Server initialization and middleware configuration, (2) Database connection and Mongoose model definition, (3) User authentication module (register, login, logout, JWT), (4) Product module (catalog, filters, CRUD), (5) Cart module, (6) Order module, (7) EJS view templates, (8) Admin panel, (9) Security hardening with Helmet.js.

2.1.4 Phase 4 – Testing (Week 9)

A comprehensive automated test suite was developed using Jest, Supertest, Puppeteer, and MongoDB Memory Server. Unit tests covered utility functions; integration tests covered all HTTP route handlers; end-to-end tests simulated complete user workflows in a real browser environment.

2.1.5 Phase 5 – Documentation (Week 10)

Project documentation including this report, inline code comments, and README files was completed. The GitHub repository was finalized and made public.

2.2 Project Planning and Timeline

Week	Phase	Activities	Deliverable
1	Requirements	Stakeholder meetings, competitive analysis, requirements documentation	Requirements doc
2	Design	Architecture, DB schema, route design, project scaffolding	Design diagrams
3	Implementation	Server setup, middleware, MongoDB connection, User model	Running server
4	Implementation	Auth module: register, login, logout, bcrypt, JWT	Auth routes live
5	Implementation	Product model, catalog routes, EJS product templates	Product catalog
6	Implementation	Cart module: add, update, remove, clear cart	Cart functional
7	Implementation	Order module: checkout, order history	Orders functional
8	Implementation	Admin panel, Helmet.js, CORS, dotenv security hardening	Secured app
9	Testing	Jest unit, Supertest integration, Puppeteer E2E tests	Test suite (53 cases)
10	Documentation	Report writing, README, code comments, GitHub finalization	This report

2.3 System Architecture

The Shouvair application is built on the three-tier architecture model:

- **Presentation Tier:** EJS templates rendered server-side by Express.js and served as complete HTML pages to the client browser. CSS stylesheets and minimal client-side JavaScript are served from the public/ directory.
- **Application Tier:** The Express.js server processes all HTTP requests, applies the middleware pipeline, routes requests to controller functions, executes business logic, and renders EJS views with data fetched from the database.
- **Data Tier:** MongoDB stores all application data (users, products, carts, orders) as BSON documents in collections. Mongoose provides schema validation, query building, and lifecycle hooks.

The internal architecture follows the Model-View-Controller (MVC) pattern:

- **Models:** Mongoose schemas and models defining the structure of data documents in MongoDB.
- **Views:** EJS templates that define the HTML structure of each page, populated with data passed from controllers.
- **Controllers:** Express.js route handler functions that receive HTTP requests, interact with models to read or write data, and pass results to views for rendering.

A complete middleware pipeline processes every incoming HTTP request before it reaches route handlers: Morgan (HTTP logging) → Helmet.js (security headers) → CORS → body-parser (request body parsing) → cookie-parser → express-session → JWT verification middleware (on protected routes) → route handler → EJS view rendering → HTTP response.

3. TOOLS AND TECHNOLOGIES

This chapter provides a detailed overview of all tools, frameworks, libraries, and technologies employed in the development of the Shouvair E-Commerce Website.

3.1 Backend Technologies

3.1.1 Node.js (v18.x LTS)

Node.js is a cross-platform, open-source JavaScript runtime environment built on Google Chrome's V8 JavaScript engine. Its non-blocking, event-driven I/O model makes it particularly efficient for data-intensive, real-time applications such as e-commerce platforms that handle numerous concurrent HTTP requests. Node.js was chosen for Shouvair because it enables the use of a single programming language (JavaScript) across the entire stack—from server logic to database queries—reducing context switching for the development team.

3.1.2 Express.js (v4.21.2)

Express.js is a minimal, unopinionated Node.js web application framework that provides a thin layer of essential web functionality including HTTP routing, middleware composition, and request/response manipulation. Express's middleware system is central to Shouvair's architecture, enabling clean composition of cross-cutting concerns (logging, security, authentication) as discrete, reusable functions in a pipeline.

3.1.3 MongoDB (v6.x) and Mongoose (v8.14.3)

MongoDB is a document-oriented NoSQL database that stores data as flexible BSON (Binary JSON) documents. Its schema-less design is well-suited to e-commerce product catalogs where different product types may have varying attributes. Mongoose is an Object Data Modeling (ODM) library that adds schema definition, type validation, instance methods, static methods, and pre/post save hooks on top of the native MongoDB driver, improving data integrity and developer ergonomics.

3.1.4 EJS (v3.1.10)

EJS (Embedded JavaScript) is a server-side templating engine that allows embedding of JavaScript expressions directly within HTML markup using `<% %>` tags. Unlike Single Page Application

frameworks, EJS renders complete HTML pages on the server, improving initial page load performance and search engine crawlability—both critical for e-commerce success.

3.1.5 Key NPM Dependencies

Package	Version	Purpose
bcrypt	6.0.0	Secure password hashing with adaptive salt rounds
jsonwebtoken	9.0.2	JWT token generation and verification for stateless auth
helmet	8.0.0	Sets 15+ HTTP security response headers automatically
morgan	1.10.0	HTTP request/response logging middleware
cors	2.8.5	Cross-Origin Resource Sharing policy enforcement
cookie-parser	1.4.7	Parses Cookie header and populates req.cookies
express-session	1.18.1	Server-side session store management
dotenv	17.2.2	Loads environment variables from .env file
body-parser	1.20.3	Parses JSON and URL-encoded request bodies
mongoose	8.14.3	MongoDB ODM with schema validation and query builder
path	0.12.7	Node.js path utility for cross-platform file paths
fs	0.0.1	File system access for reading static resources

3.2 Frontend Technologies

3.2.1 EJS Templating (Server-Side Rendering)

The frontend is implemented as a set of EJS templates that are rendered on the server and sent to the client as fully formed HTML documents. This approach avoids the complexity of a separate frontend framework while delivering dynamic, data-driven pages. EJS partials (header, footer, navbar) are included across pages for consistency and maintainability.

3.2.2 CSS

Custom CSS stylesheets in the `public/css/` directory define the visual design language of the Shouvair platform: responsive grid layouts using CSS Flexbox and Grid, product card hover effects, form styling with validation visual feedback, and a consistent color palette and typography system.

3.2.3 Client-side JavaScript

Minimal vanilla JavaScript is used for client-side interactions such as quantity steppers, filter panel toggles, and form validation feedback. No heavy client-side framework is used, keeping the application lightweight and fast.

3.3 Testing Tools

3.3.1 Jest (v30.2.0)

Jest is a comprehensive JavaScript testing framework developed by Meta. It provides a test runner, assertion library, mock functions, code coverage reporting, and snapshot testing in a single package. Jest was used for both unit tests (testing individual utility functions in isolation) and integration tests (testing HTTP routes using Supertest).

3.3.2 Supertest (v7.1.4)

Supertest is a high-level HTTP assertion library that allows sending HTTP requests to an Express application in a test environment without starting a real server. It enables clean, readable assertions on response status codes, headers, and body content for all API endpoints.

3.3.3 Puppeteer (v24.30.0)

Puppeteer is a Node.js library that provides a high-level API to control Chromium or Chrome over the DevTools Protocol. It was used for end-to-end (E2E) testing of complete user workflows (registration, product browsing, add to cart, checkout) by simulating real browser interactions.

3.3.4 MongoDB Memory Server (v10.3.0)

MongoDB Memory Server provides an in-memory MongoDB instance for testing. It allows test suites to interact with a real MongoDB API without side effects on any persistent database, and each test suite can independently set up and tear down its data state.

3.4 Development and Collaboration Tools

- Visual Studio Code – Primary IDE with ESLint, Prettier, and MongoDB extensions.
- Git & GitHub – Version control and collaborative code management. Repository: <https://github.com/dhruv276/shouvair>
- Postman – API design, manual testing, and documentation of REST endpoints during development.

- npm (v9.x) – Package management and script execution (npm start, npm test).
- dotenv – Environment variable management separating configuration from code.

4. IMPLEMENTATION

This chapter provides a detailed account of how each functional module of the Shouvair E-Commerce Website was implemented. The implementation is described module by module, covering the backend route logic, database interactions, EJS view rendering, and key code design decisions for each component.

4.1 Module 1: User Authentication

The authentication module is the foundation of the Shouvair platform, providing secure registration, login, and session management for both customers and administrators.

4.1.1 Registration (POST /auth/register)

The registration endpoint accepts the user's name, email, and password from a POST request body. Input validation checks for a valid email format and a minimum password length of 8 characters. If validation passes, the system queries MongoDB to verify that no existing user is registered with the submitted email address. The password is then hashed using bcrypt with a salt factor of 10. A new User document is created and saved to MongoDB. Upon success, a JWT is generated containing the user's `_id` and role as the payload, signed with the `JWT_SECRET` environment variable, and set in an `httpOnly` cookie with a 24-hour expiry. The user is redirected to the homepage.

4.1.2 Login (POST /auth/login)

The login endpoint retrieves the User document matching the submitted email address. If no matching user is found, a 404 response is returned. If found, `bcrypt.compare()` is used to verify the submitted password against the stored hash. On match, a new JWT is issued and set in the response cookie. On mismatch, a 401 Unauthorized response is returned. This approach ensures that timing attacks cannot distinguish between a non-existent user and an incorrect password.

4.1.3 JWT Middleware

A custom protect middleware function reads the JWT from the request cookie, verifies it using `jwt.verify()` with the `JWT_SECRET`, and attaches the decoded payload to `req.user` for downstream route handlers to access. If no token is present or verification fails, the middleware redirects to the login page. This middleware is applied to all protected routes (cart, orders, admin).

4.1.4 API Endpoints – Authentication Module

Method	Endpoint	Auth	Description
GET	/auth/login	None	Renders the login page
POST	/auth/login	None	Validates credentials, issues JWT cookie
GET	/auth/register	None	Renders the registration page
POST	/auth/register	None	Creates user, hashes password, issues JWT
GET	/auth/logout	JWT	Clears cookie, destroys session, redirects

4.2 Module 2: Product Catalog

The product catalog module provides browsing, filtering, and detailed view of footwear products, as well as administrative CRUD operations.

4.2.1 Product Listing (GET /products)

The product listing route accepts optional query string parameters (category, minPrice, maxPrice, brand, size) and dynamically constructs a Mongoose query object to filter the Product collection accordingly. Results are retrieved with Mongoose's `.find()` method and passed to the `products/index.ejs` template for rendering as a grid of product cards. The filter sidebar on the listing page submits filter selections as query strings via a GET form, allowing filtered URLs to be bookmarked and shared.

4.2.2 Product Detail (GET /products/:id)

The product detail route retrieves a single Product document by its MongoDB ObjectId. If the ID is invalid or the document is not found, a 404 response is rendered. The product data (name, description, price, images, available sizes, brand, stock) is passed to the `products/show.ejs` template, which renders the full product detail page with a size selector, quantity input, and Add to Cart / Buy Now buttons.

4.2.3 Product Schema

The Mongoose Product schema defines: name (String, required), description (String), price (Number, required), category (String, required), brand (String), sizes (Array of Numbers), images (Array of Strings – image URLs), stock (Number, default 0), ratings.average (Number), ratings.count (Number), and Mongoose timestamps (createdAt, updatedAt).

4.2.4 API Endpoints – Product Module

Method	Endpoint	Auth	Description
GET	/products	None	Lists all products with optional query filters
GET	/products/:id	None	Returns product detail page
POST	/products	Admin JWT	Creates a new product document
PUT	/products/:id	Admin JWT	Updates an existing product
DELETE	/products/:id	Admin JWT	Deletes a product document

4.3 Module 3: Shopping Cart

The cart module allows authenticated users to build a collection of products for eventual checkout. Cart state is persisted in MongoDB per user, enabling cart continuity across sessions.

4.3.1 Cart Schema

The Mongoose Cart schema contains: user (ObjectId, ref: User, unique – one cart per user), and items (Array of subdocuments each containing: product (ObjectId, ref: Product), quantity (Number), size (Number), price (Number – snapshot at time of adding)).

4.3.2 Add to Cart (POST /cart/add)

When a user clicks 'Add to Cart' on a product detail page, the route handler checks whether the product (at the selected size) already exists in the user's cart. If it does, the quantity is incremented. If not, a new cart item subdocument is pushed into the items array. The route is protected by the JWT middleware to ensure only authenticated users can access it.

4.3.3 Update and Remove

PUT /cart/update/:itemId accepts a new quantity value and updates the relevant cart item subdocument using Mongoose's positional \$ update operator. DELETE /cart/remove/:itemId uses Mongoose's \$pull operator to remove the subdocument with the matching itemId from the items array. DELETE /cart/clear sets the items array to empty, effectively clearing the cart.

4.4 Module 4: Order Management

The order management module handles the conversion of a user's cart into a confirmed order, and provides a history of all past orders.

4.4.1 Checkout (POST /orders/checkout)

The checkout route retrieves the current user's cart, validates that it contains at least one item, and accepts the shipping address from the POST body. A new Order document is created with: the user's ID, a snapshot of all cart items (product ID, quantity, size, price), the total amount calculated server-side, the shipping address, and initial status 'pending'. The order is saved to MongoDB, and the user's cart is cleared. The user is redirected to an order success page.

4.4.2 Order Schema

The Order schema includes: user (ObjectId, ref: User), items (Array: { product: ObjectId, quantity: Number, price: Number, size: Number }), totalAmount (Number), shippingAddress (Object: { street, city, state, pincode }), status (String, enum: pending/processing/shipped/delivered/cancelled, default: pending), paymentStatus (String, enum: pending/paid, default: pending), and Mongoose timestamps.

4.4.3 API Endpoints – Cart and Order Modules

Method	Endpoint	Auth	Description
GET	/cart	JWT	Renders cart page with current cart items
POST	/cart/add	JWT	Adds product (size + qty) to user's cart
PUT	/cart/update/:itemId	JWT	Updates quantity of a cart item
DELETE	/cart/remove/:itemId	JWT	Removes a specific item from the cart
DELETE	/cart/clear	JWT	Empties the entire cart
POST	/orders/checkout	JWT	Converts cart to order, clears cart
GET	/orders/my-orders	JWT	Returns authenticated user's order history
GET	/orders/:id	JWT	Returns details of a specific order
PUT	/orders/:id/status	Admin JWT	Updates order fulfillment status

4.5 Module 5: Admin Panel

The admin panel is accessible only to users with the role: 'admin' field in their User document. The adminOnly middleware function verifies that req.user.role === 'admin' after JWT verification. The admin dashboard (GET /admin) renders a management interface displaying all products in a table with edit and delete action buttons. Admins can add new products via a form at POST /admin/products, and update or delete existing products via PUT /admin/products/:id and DELETE /admin/products/:id respectively.

4.6 Security Implementation

4.6.1 Helmet.js

Helmet.js is applied as the first middleware in `server.js`, automatically setting 15+ HTTP security response headers on every response. Key headers include: `Content-Security-Policy` (restricts sources of scripts, styles, images), `X-Frame-Options: DENY` (prevents clickjacking), `X-Content-Type-Options: nosniff` (prevents MIME sniffing), `Strict-Transport-Security` (enforces HTTPS in production), and `Referrer-Policy: no-referrer`.

4.6.2 bcrypt Password Hashing

User passwords are never stored in plaintext. `bcrypt` hashes each password with a dynamically generated salt and a cost factor (salt rounds) of 10, making brute-force and rainbow table attacks computationally infeasible. `bcrypt.compare()` is used for password verification, providing timing-attack-safe comparison.

4.6.3 JWT and Cookie Security

Authentication tokens are issued as signed JWTs with a 24-hour expiry and delivered to clients exclusively via `httpOnly` cookies. `httpOnly` prevents client-side JavaScript from accessing the cookie value, mitigating Cross-Site Scripting (XSS) attacks as a vector for token theft. In production environments, the `secure` flag is set to `true`, ensuring cookies are transmitted only over HTTPS.

4.6.4 Environment Variables

All sensitive configuration values—MongoDB connection URI, JWT secret key, and session secret—are stored in a `.env` file and loaded at startup by `dotenv`. The `.env` file is listed in `.gitignore` and is therefore never committed to the GitHub repository, preventing accidental exposure of secrets.

5. MAJOR FINDINGS / OUTCOMES / OUTPUT / RESULTS

This chapter presents the key outcomes of the Shouvair project, covering the functional results achieved, the testing results, performance metrics, and the professional learnings gained during the CO-OP industrial training period.

5.1 Functional Outcomes

All core functional requirements of the Shouvair E-Commerce Website were successfully implemented and verified. The following table summarizes the functional outcomes against each module:

Module	Feature Delivered	Status	Test Coverage
User Authentication	Registration, login, JWT auth, logout, bcrypt hashing	Complete	95%
Product Catalog	Listing, filtering, detail page, admin CRUD	Complete	92%
Shopping Cart	Add, update quantity, remove item, clear cart	Complete	89%
Order Management	Checkout, order placement, order history, status update	Complete	87%
Admin Panel	Product management dashboard, role-based access	Complete	85%
Security	Helmet.js, CORS, JWT httpOnly cookies, dotenv	Complete	N/A
Testing Suite	53 test cases: unit, integration, E2E	Complete	~92%

The application runs as expected on Node.js v18.x with MongoDB v6.x. The GitHub repository (<https://github.com/dhruv276/shouvair>) contains all source files, test suites, and configuration files. The EJS-rendered pages load complete, styled HTML from the server with all dynamic data correctly populated from the MongoDB database.

5.2 Testing Results

The testing phase was conducted over one week using a multi-layer testing strategy. The following test suites were executed:

5.2.1 Unit Tests – Utility Functions

Unit tests were written for the auth.js JWT utility (generateToken, verifyToken) and the validators.js input validation functions. Key findings:

- generateToken() correctly produces a signed JWT string for any given user payload.
- verifyToken() successfully decodes valid tokens and throws appropriate errors (JsonWebTokenError, TokenExpiredError) for invalid or expired tokens.
- Email validation correctly accepts all RFC 5322 compliant formats and rejects invalid patterns.
- Password length validation enforces the 8-character minimum consistently.

5.2.2 Integration Tests – HTTP Route Handlers

Integration tests used Supertest to send HTTP requests to the Express application connected to MongoDB Memory Server. Key findings:

- All authentication endpoints (register, login, logout) returned expected status codes and set/cleared cookies correctly.
- Duplicate email registration returned 400 as expected.
- Unauthenticated access to protected cart and order routes correctly returned 401 or redirected to login.
- Product filter queries (category, price range) returned correctly filtered result sets from the in-memory database.
- Admin-only endpoints correctly returned 403 for non-admin JWT holders.

5.2.3 End-to-End Tests – Browser Automation (Puppeteer)

Puppeteer tests executed complete user workflows in a headless Chromium browser. Key findings:

- User registration flow: form submission → successful redirect to homepage → navbar displays logged-in username. All assertions passed.
- Product browsing and add-to-cart flow: login → browse catalog → select product → choose size → add to cart → navigate to cart → product visible with correct size and quantity.
- Logout flow: click logout → redirect to login page → protected routes return to login redirect.

5.2.4 Test Results Summary

Test Suite	Written	Passed	Failed	Coverage
Auth Utility Unit Tests	8	8	0	100%
Validator Utility Unit Tests	12	12	0	100%
Auth Route Integration Tests	7	7	0	95%
Product Route Integration Tests	9	9	0	92%
Cart Route Integration Tests	8	8	0	89%
Order Route Integration Tests	6	6	0	87%
Registration E2E Test	1	1	0	N/A
Shopping Flow E2E Test	1	1	0	N/A
Logout E2E Test	1	1	0	N/A
TOTAL	53	53	0	~92%

5.3 Performance Metrics

Basic performance testing was conducted by measuring server response times under simulated load using Postman and browser DevTools. The following metrics were observed:

Metric	Target	Achieved	Status
Homepage average response time	< 2000ms	~180ms	PASS
Product listing page response time	< 2000ms	~230ms	PASS
Cart add operation response time	< 2000ms	~95ms	PASS
Login (bcrypt comparison) time	< 500ms	~155ms	PASS
JWT generation time	Negligible	~2ms	PASS
Full Jest test suite execution	< 60 seconds	~42 seconds	PASS
Overall code coverage	> 80%	~92%	PASS

5.4 Learnings from Industry

The CO-OP industrial training period provided invaluable exposure to real-world software development practices beyond what is possible in a purely academic environment. Key learnings are summarized below:

5.4.1 Technical Learnings

- Deepened practical understanding of the Node.js event loop, asynchronous programming with `async/await`, and error handling in asynchronous contexts.
- Gained hands-on experience designing and querying MongoDB schemas for complex, interrelated data (users, products, carts, orders).
- Learned to configure and compose a production-grade Express.js middleware pipeline, understanding the order-sensitivity of middleware (why `Helmet` must precede `body-parser`, etc.).
- Understood the practical implications of security decisions: the difference between cookie storage options (`httpOnly` vs. `standard`), the cost factor in `bcrypt`, and the payload size constraints of JWTs.
- Developed proficiency in test-driven thinking: writing tests before or alongside implementation code, and using MongoDB Memory Server to isolate test environments.
- Learned to use Puppeteer for realistic E2E test automation, including handling asynchronous page navigation and dynamic element waiting strategies.

5.4.2 Professional and Collaborative Learnings

- Practised collaborative Git workflows: feature branching, pull requests, code reviews, and resolving merge conflicts in a shared repository.
- Developed skills in writing clear API documentation and inline code comments for maintainability.
- Experienced the discipline of breaking a large system into independently testable, deployable modules rather than building everything monolithically.
- Gained exposure to how industry teams structure Node.js projects (directory conventions, configuration management, environment-based settings).

Skill Area	Before CO-OP	After CO-OP
Node.js / Express.js	Beginner	Proficient – built production-grade middleware pipeline
MongoDB / Mongoose	Beginner	Proficient – designed schemas, queries, and relationships
JWT / bcrypt Security	Awareness only	Hands-on implementation in a real system

Automated Testing	Minimal exposure	Wrote 53 tests across unit, integration, E2E layers
Git Collaboration	Personal projects only	Multi-developer branching and PR workflow
API Design (REST)	Theoretical	Designed and implemented complete REST API
Project Documentation	Basic	Produced full CO-OP project report

6. CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

The Shouvair E-Commerce Website project successfully achieved all defined objectives, delivering a fully functional, secure, and well-tested full-stack web application for online footwear retail. The project was executed over a ten-week CO-OP industrial training period by Aarav Shourie (2210990010), Dhruv Kundrai (2210990276), Aarya Rishi (2210990012), and Advitya Hans (2210990076), students of Computer Science and Engineering at Chitkara University.

The application demonstrates a complete implementation of the MVC architectural pattern using Node.js, Express.js, MongoDB, and EJS—a technology stack that is widely used in modern industry for building scalable, maintainable web applications. The security implementation, encompassing bcrypt password hashing, JWT-based stateless authentication, Helmet.js HTTP header hardening, and environment-based secret management, reflects industry-standard best practices.

The testing strategy—using Jest for unit and integration tests, Supertest for HTTP route assertions, Puppeteer for end-to-end browser automation, and MongoDB Memory Server for test isolation—resulted in a 53-test suite with a 100% pass rate and approximately 92% code coverage. This represents a comprehensive approach to software quality assurance that goes beyond typical student project standards.

From an industry learning perspective, this CO-OP project provided the team with deeply practical experience in full-stack JavaScript development, collaborative Git workflows, REST API design, database schema modeling, and software testing—skills that are directly applicable and in high demand in the current software engineering job market.

The complete source code is publicly available on GitHub at <https://github.com/dhruv276/shouvair>, and the project is structured in a manner that enables straightforward deployment to cloud hosting platforms with minimal additional configuration.

6.2 Future Scope

While the current version of Shouvair meets all defined project objectives, several enhancements are planned or recommended for future iterations to evolve the platform into a production-ready commercial product:

6.2.1 Payment Gateway Integration

The most critical next enhancement is the integration of a real payment gateway. Razorpay (India-focused) or Stripe (global) are the recommended options. This would involve creating payment initiation routes, handling payment status webhooks from the gateway, updating the Order document's `paymentStatus` field upon confirmed payment, and displaying payment confirmation to the user.

6.2.2 Email Notification System

Transactional email notifications—order confirmation, shipping update, delivery confirmation, and password reset emails—would significantly improve the user experience. Nodemailer with a cloud email delivery service (SendGrid or AWS SES) is the recommended implementation approach.

6.2.3 Cloud Image Storage

Currently, product images are referenced by URL. Enabling direct image uploads from the admin panel and storing them on a cloud service such as AWS S3 or Cloudinary would allow complete product management within the platform, without dependency on external image hosting.

6.2.4 React.js Frontend Migration

Migrating the EJS server-side rendered frontend to a React.js Single Page Application (with the Express server repurposed as a pure JSON REST API) would enable a significantly richer user experience: instant search results without page reloads, real-time cart updates, skeleton loading states, and smoother client-side navigation.

6.2.5 Advanced Analytics Dashboard

An analytics dashboard for admins showing key business metrics—total revenue, orders per day, best-selling products, customer acquisition trends, and cart abandonment rates—would transform Shouvair from a transactional platform into a business intelligence tool for footwear retailers.

6.2.6 Product Reviews and Ratings

A user review system allowing verified purchasers to submit written reviews and star ratings would add social proof and improve purchase decision confidence for new shoppers. The ratings.average and ratings.count fields are already present in the Product schema in anticipation of this feature.

6.2.7 Progressive Web App (PWA)

Converting the Shouvair web application into a Progressive Web App (PWA) by adding a service worker, manifest.json, and offline-capable caching strategies would allow users to install the app on their mobile home screens and browse the product catalog even with limited connectivity.

6.2.8 Containerization and CI/CD Pipeline

Dockerizing the application (with separate containers for the Node.js server and MongoDB) and setting up a Continuous Integration / Continuous Deployment (CI/CD) pipeline using GitHub Actions would automate testing on every commit and enable one-command deployment to cloud platforms such as AWS EC2, Google Cloud Run, or Railway.

REFERENCES

- [1] OpenJS Foundation. (2024). Node.js v18.x Documentation. Retrieved from <https://nodejs.org/en/docs/>
- [2] Express.js Community. (2024). Express 4.x API Reference. Retrieved from <https://expressjs.com/en/4x/api.html>
- [3] MongoDB Inc. (2024). MongoDB Manual v6.0. Retrieved from <https://www.mongodb.com/docs/manual/>
- [4] Mongoose ODM. (2024). Mongoose v8.x API Documentation. Retrieved from <https://mongoosejs.com/docs/>
- [5] npm Inc. (2024). bcrypt Package. Retrieved from <https://www.npmjs.com/package/bcrypt>
- [6] npm Inc. (2024). jsonwebtoken Package. Retrieved from <https://www.npmjs.com/package/jsonwebtoken>
- [7] npm Inc. (2024). helmet Package. Retrieved from <https://www.npmjs.com/package/helmet>
- [8] npm Inc. (2024). morgan Package. Retrieved from <https://www.npmjs.com/package/morgan>
- [9] Auth0. (2024). Introduction to JSON Web Tokens. Retrieved from <https://jwt.io/introduction>
- [10] OWASP Foundation. (2023). OWASP Top Ten – 2021. Retrieved from <https://owasp.org/www-project-top-ten/>
- [11] Jest Authors. (2024). Jest Testing Framework Documentation. Retrieved from <https://jestjs.io/docs/getting-started>
- [12] Supertest Authors. (2024). Supertest on GitHub. Retrieved from <https://github.com/ladjs/supertest>
- [13] Puppeteer Team. (2024). Puppeteer API Documentation. Retrieved from <https://pptr.dev/>
- [14] Typegoose / mongodb-memory-server. (2024). mongodb-memory-server Documentation. Retrieved from <https://github.com/typegoose/mongodb-memory-server>
- [15] EJS Community. (2024). EJS Documentation. Retrieved from <https://ejs.co/>
- [16] Google. (2024). Core Web Vitals. Retrieved from <https://web.dev/vitals/>

- [17] Casciaro, M., & Mammino, L. (2020). Node.js Design Patterns (3rd ed.). Packt Publishing.
- [18] Haverbeke, M. (2018). Eloquent JavaScript (3rd ed.). No Starch Press.
- [19] NASSCOM. (2023). Indian E-Commerce Industry Report 2023. NASSCOM Publications.
- [20] Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures (Doctoral dissertation, University of California, Irvine).

APPENDICES

Appendix A: GitHub Repository

The complete source code for the Shouvair E-Commerce Website is available at the following public GitHub repository:

URL: <https://github.com/dhruv276/shouvair>

The repository contains: all Node.js source files (server.js, route handlers, Mongoose models, EJS templates, CSS stylesheets), the full automated test suite (tests/ directory), package.json with all dependencies, .gitignore (excluding node_modules and .env), and README.md with setup instructions.

Appendix B: Key Source Code Snippets

B.1 – Server Entry Point: Middleware Configuration

```
const express = require('express');
const helmet = require('helmet');
const morgan = require('morgan');
const mongoose = require('mongoose');
require('dotenv').config();
const app = express();
app.set('view engine', 'ejs');
app.use(helmet());
app.use(morgan('dev'));
app.use(require('cors')({ credentials: true }));
app.use(express.json());
app.use(express.urlencoded({ extended: true }));
app.use(require('cookie-parser')());
app.listen(process.env.PORT || 3000);
```

B.2 – JWT Authentication Middleware

```
const protect = (req, res, next) => {
  const token = req.cookies.jwt;
  if (!token) return res.redirect('/auth/login');
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch (e) { res.redirect('/auth/login'); }
};
```

B.3 – Product Mongoose Schema

```
const productSchema = new mongoose.Schema({
  name:      { type: String, required: true },
  price:     { type: Number, required: true },
  category:  { type: String, required: true },
  brand:     String,
  sizes:     [Number],
  images:    [String],
  stock:     { type: Number, default: 0 },
}, { timestamps: true });
```

Appendix C: Team Contributions

Team Member	Roll No.	Primary Contributions
Aarav Shourie	2210990010	Server architecture, auth module (JWT/bcrypt), security hardening (Helmet.js, CORS), overall integration
Dhruv Kundrai	2210990276	Product catalog module, MongoDB schema design, admin panel, GitHub repository management
Aarya Rishi	2210990012	Shopping cart and order modules, EJS template development, CSS styling and responsive design
Advitya Hans	2210990076	Test suite development (Jest, Supertest, Puppeteer), MongoDB Memory Server setup, project documentation